

HW4 B1121123 巫煊崙

程式碼

```
'timescale 1ns/10ps
'define WL 9
'include "Hw4_3bitCLA_rtl - Student.v"

module Hw4_9bitCLA (
    input ['WL-1:0] X,
    input ['WL-1:0] Y,
    input Cin,
    output ['WL-1:0] Sum,
    output Cout,
    output OV
);

    wire [2:0] G_LCG;
    wire [2:0] P_LCG;
    wire [3:0] Carry_in;

    assign Carry_in[0] = Cin;
    assign Carry_in[1] = G_LCG[0] | (P_LCG[0] & Carry_in[0]);
    assign Carry_in[2] = G_LCG[1] | (P_LCG[1] & G_LCG[0]) | (P_LCG[1] & P_LCG[0] & Carry_in[0]);
```

```
    assign Carry_in[3] = G_LCG[2] | (P_LCG[2] & G_LCG[1]) | (P_LCG[2] & P_LCG[1] & G_LCG[0]) | (P_LCG[2] &
P_LCG[1] & P_LCG[0] & Carry_in[0]);
    assign Cout = Carry_in[3];
```

```
Hw4_3bitCLA CLA_1 (
    .X(X[2:0]),
    .Y(Y[2:0]),
    .Cin(Carry_in[0]),
    .Sum(Sum[2:0]),
    .G(G_LCG[0]),
    .P(P_LCG[0])
);
```

```
Hw4_3bitCLA CLA_2 (
    .X(X[5:3]),
    .Y(Y[5:3]),
    .Cin(Carry_in[1]),
    .Sum(Sum[5:3]),
    .G(G_LCG[1]),
    .P(P_LCG[1])
);
```

```
Hw4_3bitCLA CLA_3 (
```

```

        .X(X[8:6]),
        .Y(Y[8:6]),
        .Cin(Carry_in[2]),
        .Sum(Sum[8:6]),
        .G(G_LCG[2]),
        .P(P_LCG[2])
    );

```

```

// OV = (X_sign == Y_sign) && (Sum_sign != X_sign)
assign OV = (X['WL-1] ~^ Y['WL-1]) & (Sum['WL-1] ^ X['WL-1]);

```

```

endmodule

```

```

`timescale 1ns/10ps
`include "Hw4_1bitCLA_rtl - Student.v"
module Hw4_3bitCLA (
    input [2:0] X,
    input [2:0] Y,
    input Cin,
    output [2:0] Sum,
    output G, // Group Generate
    output P // Group Propagate

```

```

);

wire [2:0] G_LCG;    // 3-bit lookahead carry generate from sub-modules
wire [2:0] P_LCG;    // 3-bit lookahead carry propagate from sub-modules
wire [2:0] Carry_in;

assign Carry_in[0] = G_LCG[0] | (P_LCG[0] & Cin);
assign Carry_in[1] = G_LCG[1] | (P_LCG[1] & G_LCG[0]) | (P_LCG[1] & P_LCG[0] & Cin);
assign Carry_in[2] = G_LCG[2] | (P_LCG[2] & G_LCG[1]) | (P_LCG[2] & P_LCG[1] & G_LCG[0]) | (P_LCG[2] & P_LCG[1]
& P_LCG[0] & Cin);

Hw4_1bitCLA cla_1 (
    .X(X[0]),
    .Y(Y[0]),
    .Cin(Cin),
    .Sum(Sum[0]),
    .G(G_LCG[0]),
    .P(P_LCG[0])
);

Hw4_1bitCLA cla_2 (

```

```
.X(X[1]),  
.Y(Y[1]),  
.Cin(Carry_in[0]),  
.Sum(Sum[1]),  
.G(G_LCG[1]),  
.P(P_LCG[1])  
);
```

```
Hw4_1bitCLA cla_3 (  
.X(X[2]),  
.Y(Y[2]),  
.Cin(Carry_in[1]),  
.Sum(Sum[2]),  
.G(G_LCG[2]),  
.P(P_LCG[2])  
);
```

```
assign P = P_LCG[2] & P_LCG[1] & P_LCG[0];  
assign G = G_LCG[2] | (P_LCG[2] & G_LCG[1]) | (P_LCG[2] & P_LCG[1] & G_LCG[0]);
```

```
endmodule
```

```
`timescale 1ns/10ps
module Hw4_1bitCLA (
    input X,
    input Y,
    input Cin,
    output Sum,
    output G,
    output P
);

assign P = X ^ Y;
assign G = X & Y;
assign Sum = P ^ Cin; //  $X \oplus Y \oplus Cin$ 

endmodule
```

模擬結果

```
2. 163.25.97.100 (stu004)
of a written license agreement with Synopsys, Inc. All other use, reproduction,
or distribution of this software is strictly prohibited. Licensed Products
communicate with Synopsys servers for the purpose of providing software
updates, detecting software piracy and verifying that customers are using
Licensed Products in conformity with the applicable License Key for such
Licensed Products. Synopsys will use information gathered in connection with
this process to deliver software updates and pursue software pirates and
infringers.

Inclusivity & Diversity - Visit SolvNetPlus to read the "Synopsys Statement on
Inclusivity and Diversity" (Refer to article 000036315 at
https://solvnetplus.synopsys.com)

The design hasn't changed and need not be recompiled.
If you really want to, delete file simv.daidir/.vcs.timestamp and
run VCS again.

Chronologic VCS simulator copyright 1991-2022
Contains Synopsys proprietary information.
Compiler version T-2022.06_Full64; Runtime version T-2022.06_Full64; May 26 08:16 2026
*Verdi* Loading libsscore_vcs202206.so
FSDB Dumper for VCS, Release Verdi_T-2022.06, Linux x86_64/64bit, 05/29/2022
(C) 1996 - 2022 by Synopsys, Inc.
*Verdi* FSDB WARNING: The FSDB file already exists. Overwriting the FSDB file may crash the programs that are using this file.
*Verdi* : Create FSDB file 'HW4_CLA_RTb.fsdb'
*Verdi* : Begin traversing the scope (CLA_RTb), layer (0).
*Verdi* : End of traversing.

// Congratulations!! Your Verilog Code is correct!!

$finish called from file "Hw4_CLA_rtb.v", line 67.
$finish at simulation time 2621480000
V C S S i m u l a t i o n R e p o r t
Time: 2621480000 ps
CPU Time: 0.400 seconds; Data structure size: 0.0Mb
Tue May 26 08:16:56 2026
CPU time: .441 seconds in simulation
[stu004@vlsi LAB4]$
```

波形圖

